Formal Verification: Safety Properties and Model Checking

Formal Verification: Safety Properties and Model Checking I

This section establishes safety properties (`sec:safety-properties`), proves invariant preservation lemmas (`sec:invariant-lemmas`), demonstrates liveness guarantees (`sec:liveness-properties`), derives complexity bounds (`sec:complexity-bounds`), and presents model checking verification (`sec:model-checking`).

Safety Properties

Theorem (Belief Injection Resistance)

Under CIF with firewall detection rate r_f and sandboxing verification rate r_s:

$$P(\mathcal{A}_{BI} \text{ succeeds}) \leq (1 - r_f) \cdot (1 - r_s) \tag{1}$$

Proof.

We prove this theorem by analyzing the sequential defense mechanism and applying probability theory for independent events.

Setup: Let ϕ_{adv} be an adversarial belief that the attacker attempts to inject into agent a_i's verified belief set $\mathcal{B}_{verified}$.

Defense Model: The CIF implements two sequential defenses:

1. **Cognitive Firewall** $\mathcal{F}$: Classifies incoming messages as ACCEPT, QUARANTINE, or REJECT with detection rate r_f
2. **Belief Sandbox**: Verifies quarantined beliefs before promotion with verification rate r_s

Step 1: For ϕ_{adv} to enter $\mathcal{B}_{verified}$, it must first pass the firewall. Let $E_f =$ "Firewall fails to detect ϕ_{adv}". By definition of detection rate:

$$P(E_f) = 1 - r_f \tag{2}$$

Step 2: If ϕ_{adv} passes the firewall (event E_f), it enters $\mathcal{B}_{provisional}$. For injection to succeed, it must then pass sandbox verification.
Let $E_s =$ "Sandbox fails to detect ϕ_{adv}". By definition of verification rate:

$$P(E_s) = 1 - r_s \tag{3}$$

Corollary (Empirical Security Bound)

With $r_f = 0.8$ and $r_s = 0.7$:

$$P(\textit{success}) \leq (1 - 0.8)(1 - 0.7) = 0.2 \cdot 0.3 = 0.06 \tag{6}$$

Corollary (Layered Defense Generalization)

For n independent defense layers with rates $r_1, \ldots, r_n$:

$$P(\textit{success}) \leq \prod_{i=1}^{n} (1 - r_i) \tag{7}$$

No Trust Amplification (Model-Checking Target) I

Theorem (No Trust Amplification)

For any path $p = (a_0, a_1, \ldots, a_k)$ in the communication graph:

$$\mathcal{T}^{path}_{a_0 \to a_k} \leq \min_{i \in [0, k-1]} \mathcal{T}_{a_i \to a_{i+1}} \tag{8}$$

Proof.

By trust delegation rule (`def:trust-delegation`) and induction on path length.

Base case $(k = 1)$:

$$\mathcal{T}_{a_0 \to a_1} = \mathcal{T}_{a_0 \to a_1} \quad \checkmark \tag{9}$$

Inductive step: Assume the theorem holds for paths of length k. For path length $k + 1$:

$$\begin{aligned} \mathcal{T}_{a_0 \to a_{k+1}} &= \min(\mathcal{T}^{path}_{a_0 \to a_k}, \mathcal{T}_{a_k \to a_{k+1}}) \cdot \delta \\ &\leq \min(\min_{i \in [0, k-1]} \mathcal{T}_{a_i \to a_{i+1}}, \mathcal{T}_{a_k \to a_{k+1}}) \\ &= \min_{i \in [0, k]} \mathcal{T}_{a_i \to a_{i+1}} \end{aligned} \tag{10}$$

□

Goal Alignment Preservation I

Theorem (Goal Alignment Invariant)

If the system starts with aligned goals and all goal updates follow the delegation protocol:

$$\textit{Aligned}(\mathcal{G}_i^0) \land \forall t : \textit{ValidUpdate}(\mathcal{G}_i^t, \mathcal{G}_i^{t+1}) \Rightarrow \forall t : \textit{Aligned}(\mathcal{G}_i^t) \tag{11}$$

Proof.

ValidUpdate requires new goals derive from principal or valid delegation chain. By induction on time t, alignment is preserved at every step. □

Invariant Preservation Lemmas

Lemma (Belief Consistency Preservation)

If $\mathcal{B}_i^t$ is consistent and the belief update follows Rule B-DIRECT or B-DELEGATED (`sec:belief-update-rules`), then $\mathcal{B}_i^{t+1}$ is consistent.

Invariant Preservation Lemmas II

Proof.

Let Consistent($\mathcal{B}$) denote that no high-confidence contradictions exist:

$$\text{Consistent}(\mathcal{B}) \iff \nexists \phi, \psi : \mathcal{B}(\phi) > \tau \wedge \mathcal{B}(\psi) > \tau \wedge (\phi \wedge \psi \vdash \bot) \tag{12}$$

Case 1: Rule B-DIRECT applies.
The update adds evidence for proposition ϕ:

$$\mathcal{B}_i^{t+1}(\phi) = \text{BayesUpdate}(\mathcal{B}_i^t(\phi), c \cdot \mathcal{T}_{i \to s}) \tag{13}$$

If the update would create contradiction (i.e., both $\mathcal{B}^{t+1}(\phi) > \tau$ and $\mathcal{B}^{t+1}(\psi) > \tau$ for contradictory ϕ, ψ), the sandbox consistency check in Rule S-PROMOTE rejects promotion:

$$V(\pi) = 1 \wedge \text{Consistent}(\mathcal{B}_{verified} \cup \{\phi\}) \wedge |\text{Corroborate}(\phi)| \geq \kappa \tag{14}$$

Therefore, only consistent updates reach $\mathcal{B}_{verified}$.
Case 2: Rule B-DELEGATED applies.
Same argument, with additional trust decay ensuring lower confidence for delegated evidence. □

Lemma (Trust Matrix Preservation)

The trust matrix $\mathcal{T}$ remains well-formed after any valid update:

$$\forall i, j : 0 \leq \mathcal{T}_{i \to j} \leq 1 \tag{15}$$

Proof.

By `def:trust-function`, trust is computed as:

$$\mathcal{T}^t_{i \to j} = \alpha \cdot T_{base}(j) + \beta \cdot T^t_{rep}(j) + \gamma \cdot T^t_{ctx}(i, j) \tag{16}$$

where $\alpha + \beta + \gamma = 1$ and each component $T_* \in [0, 1]$.
Therefore:

$$\mathcal{T}^t_{i \to j} \in [\min(T_{base}, T_{rep}, T_{ctx}), \max(T_{base}, T_{rep}, T_{ctx})] \subseteq [0, 1] \tag{17}$$

For delegation (`def:trust-delegation`):

$$\mathcal{T}^{del}_{i \to k} = \min(\mathcal{T}_{i \to j}, \mathcal{T}_{j \to k}) \cdot \delta^d \tag{18}$$

Since $\min(\cdot) \leq 1$ and $\delta \in (0, 1)$, we have $\mathcal{T}^{del}_{i \to k} \in [0, 1]$. □

Lemma (Provenance Chain Integrity)

Every belief in $\mathcal{B}_{verified}$ has a valid, verifiable provenance chain.

Proof.

By Rule S-PROMOTE (`sec:sandbox-rules`), promotion from $\mathcal{B}_{provisional}$ to $\mathcal{B}_{verified}$ requires:

$$V(\pi(\phi)) = 1 \tag{19}$$

where V is the provenance verification function.
By construction of the sandbox, beliefs can only enter $\mathcal{B}_{verified}$ through:

1. Initial system beliefs (hardcoded with SYSTEM_VERIFIED provenance)
2. Promotion from provisional (verified by V)

In both cases, provenance is verified. By induction on the history of $\mathcal{B}_{verified}$, all beliefs have valid provenance. □

Lemma (Permission Boundary Preservation)

Effective permissions never exceed granted permissions:

$$\forall a_i, \forall \textit{action} : \mathcal{P}_{effective}(a_i, \textit{action}) \leq \mathcal{P}_{role}(a_i, \textit{action}) \tag{20}$$

Proof.

By `def:permission-layer`:

$$\mathcal{P}_{effective}(a_i) = \mathcal{P}_{role}(a_i) \cap \mathcal{P}_{delegated}(a_i) \cap \mathcal{P}_{context}(a_i) \tag{21}$$

Since intersection can only reduce permissions ($A \cap B \cap C \subseteq A$), we have $\mathcal{P}_{effective}(a_i, \text{action}) \leq \mathcal{P}_{role}(a_i, \text{action})$ for all actions. □

Lemma (Firewall Completeness)

Every incoming message is classified by the firewall.

Proof.

By `def:firewall`, the firewall is a total function:

$$\mathcal{F} : \mathcal{M} \to \{\text{ACCEPT}, \text{QUARANTINE}, \text{REJECT}\} \tag{22}$$

The decision rules cover all cases:

- If $D_{inj}(m) > \tau_1$: REJECT
- Else if $D_{sus}(m) > \tau_2$: QUARANTINE
- Else: ACCEPT

Since D_{inj} and D_{sus} are defined for all messages, and the conditions are exhaustive, every message receives a classification. $\square$

Liveness Properties

Theorem (Firewall Liveness)

CIF firewall preserves liveness for legitimate inputs:

$$\forall m \in \mathcal{M}_{legitimate} : P(\mathcal{F}(m) = \mathit{ACCEPT}) \geq 1 - \epsilon_{fp} \tag{23}$$

Proof.

By firewall design, false positive rate is bounded by ϵ_{fp}. Legitimate messages are rejected only on false positive, establishing the bound. □

Theorem (Byzantine Consensus Termination)

With $n \geq 3f + 1$ agents and at most f Byzantine:

$$P(\textit{consensus reached in } O(f+1) \textit{ rounds}) = 1 \tag{24}$$

Proof.

Standard Byzantine agreement result (Lamport et al., 1982). With honest majority $n \geq 3f + 1$, the PBFT protocol guarantees termination in $f + 1$ rounds. □

Complexity Bounds

Space Complexity I

Space Complexity I

Table 1: Per-component space complexity.

Component	Space	Notes
Belief state	$O(\|\Phi\|)$	Propositions tracked
Provenance	$O(\|\Phi\| \cdot d)$	d = max chain depth
Trust matrix	$O(n^2)$	Pairwise trust
Tripwires	$O(k)$	k = canary count
History	$O(w)$	Window size

Theorem (Total Space Bound)

The total space complexity of CIF for n agents with $|\Phi|$ propositions, provenance depth d, k tripwires, and window size w is:

$$S_{total} = O(n \cdot (|\Phi| \cdot d + k + w) + n^2) \tag{25}$$

Proof.
Per agent:

- Belief state: $|\Phi|$ propositions with confidence values $= O(|\Phi|)$
- Provenance: Each belief has chain of depth at most $d = O(|\Phi| \cdot d)$
- Tripwires: k canary beliefs $= O(k)$
- History window: w events $= O(w)$

Total per agent: $O(|\Phi| \cdot d + k + w)$
Global:

- Trust matrix: $n \times n = O(n^2)$
- Shared state: $O(|\mathcal{S}|)$ (constant relative to n)

Total for n agents: $O(n \cdot (|\Phi| \cdot d + k + w) + n^2)$. □

Time Complexity I

Time Complexity I

Table 2: Per-operation time complexity.

Operation	Time	Frequency
Firewall check	$O(\|m\|)$	Per message
Trust update	$O(1)$	Per interaction
Drift detection	$O(\|\Phi\|)$	Per window
Consensus	$O(n^2)$	Per decision
Provenance trace	$O(d)$	On demand

Theorem (Per-Message Processing Time)

Processing a single message m takes time:

$$T_{msg} = O(|m| + \min(d, \textit{timeout})) \tag{26}$$

Proof.

Message processing pipeline:

1. Firewall classification: $O(|m|)$ for pattern matching and semantic analysis
2. Sandbox entry (if quarantined): $O(1)$
3. Provenance verification (if promoted): $O(d)$ to trace chain
4. Trust update: $O(1)$ weighted average
5. Tripwire check: $O(k)$, typically $k \ll |m|$

Provenance verification can be bounded by timeout. Total: $O(|m| + d)$. □

Theorem (Consensus Round Complexity)

Byzantine consensus requires $O((f+1) \cdot n^2)$ message complexity.

Proof.

Standard PBFT result:

- $f+1$ rounds required to guarantee termination with f Byzantine failures
- Each round requires all-to-all communication: $O(n^2)$ messages
- Total: $O((f+1) \cdot n^2)$

□

Latency Overhead I

Theorem (Bounded Latency Overhead)

CIF adds latency:

$$L_{CIF} = L_{firewall} + L_{sandbox} \cdot P(\textit{quarantine}) + L_{verify} \cdot P(\textit{verify}) \tag{27}$$

Latency Overhead II

Proof.

Expected latency is sum of:

1. Firewall (always): $L_{firewall}$
2. Sandbox (conditional): $L_{sandbox} \cdot P(\text{message quarantined})$
3. Verification (conditional): $L_{verify} \cdot P(\text{belief promoted})$

With illustrative parameters (these are worked-example values chosen to exercise the bound, not measurements; Part 2 reports a mean firewall latency of 0.08 ms per sample on its prototype pipeline, and the deployment-scale constants below are not derived from it):

- $L_{firewall} = 10\text{ms}$
- $L_{sandbox} = 5\text{ms}$, $P(\text{quarantine}) = 0.3$
- $L_{verify} = 15\text{ms}$, $P(\text{verify}) = 0.2$

$$L_{CIF} \approx 10 + 5 \cdot 0.3 + 15 \cdot 0.2 = 10 + 1.5 + 3 = 14.5\text{ms} \tag{28}$$

Compared to baseline $\approx 11.8\text{ms}$: overhead factor ≈ 1.23 (23%). □

Formal Model Checking

State Space Definition I

Definition (System State)

The complete system state is the tuple:

$$s = (\sigma_1, \ldots, \sigma_n, \mathcal{S}, \mathcal{T}) \tag{29}$$

where σ_i is agent i's cognitive state, $\mathcal{S}$ is shared state, and $\mathcal{T}$ is the trust matrix.

We verify the following CTL (Computation Tree Logic) properties:

Property (Safety: Consensus Integrity)

$$AG(\neg \textit{compromised}(\mathcal{B}_{consensus})) \tag{30}$$

"Always globally, consensus beliefs are not compromised."

Property (Liveness: Request-Response)

$$AG(\textit{request} \Rightarrow AF(\textit{response})) \tag{31}$$

"Every request eventually gets a response."

Property (Fairness: Tripwire Checking)

$$AG(AF(\textit{tripwire\_checked})) \tag{32}$$

"Tripwires are checked infinitely often."

Model Checking Results I

The following table summarizes the expected state space exploration for each property based on formal analysis of the CIF specification. These values represent theoretical bounds derived from the state space definition (`def:system-state-verification`) and complexity analysis (`sec:complexity-bounds`). Part 2 carries the executable NuSMV, SPIN and TLA+ specifications and the script that runs them, but records in `output/formal/verification_summary.json` that none of the three checkers was present in that environment, so no part of this series reports an executed model-checking verdict.

Model Checking Results I

Table 3: Model checking state-space bounds. "Verified" here denotes a theoretical bound established in this part, not a checker's verdict; Part 2's S04 carries the executable NuSMV/SPIN/TLA+ specifications, and records that no checker was available to run them. No model checker was executed anywhere in this series.

Property	Status	Reference
Belief integrity	theoretical bound; executable run in Part 2 §S04	`thm:belief-injection`
Trust bounded	theoretical bound; executable run in Part 2 §S04	`thm:trust-bound`
No deadlock	theoretical bound; executable run in Part 2 §S04	`thm:firewall-liveness`
Eventual detection	theoretical bound; executable run in Part 2 §S04	`thm:progressive-detection`

Note: *Model checking tool configurations (NuSMV, SPIN, TLA+) and verification parameters are provided in Part 2: Computational Validation, which presents the executable implementations and empirical verification results.*

Verification Results Summary I

The following table records which property each tool is configured to check and the status of the corresponding analytical result. It is a plan, not a set of outcomes: no model-checking run is executed in this paper. These guarantees follow from the CTL/LTL property specifications (`sec:temporal-properties`) applied to the state space definition (`sec:state-space`). Empirical execution of these verification configurations, including runtime measurements and counterexample analysis, is presented in Part 2.

Verification Results Summary I

Table 4: Model-checking plan: the property each tool is configured to check, and the status of the corresponding analytical result. No run of these configurations is reported in this paper; execution is Part 2's.

Tool	Property to check	Status of the analytical result	Reference
NuSMV	Belief Integrity	Proved (S01)	`thm:belief-injection`
NuSMV	No Trust Amplification	Proved (S01)	`thm:trust-bound`
SPIN	No Deadlock	Proved (S01)	`thm:firewall-liveness`
SPIN	Eventual Detection	Asserted, proof deferred	`thm:progressive-detection`
TLA+	Type Invariant	Definitional	`def:system-state`
TLA+	Consensus Integrity	Proved (S01)	`thm:byzantine-termination`

Counterexample Analysis I

When verification fails, model checkers produce counterexamples [**?**]. Analysis procedure:

Counterexample Analysis I

1. **Extract trace**: Sequence of states leading to violation
2. **Identify trigger**: First state where invariant fails
3. **Root cause**: Determine which transition violated property
4. **Fix**: Strengthen preconditions or add defense mechanism
5. **Re-verify**: Confirm fix resolves violation

Example: Counterexample Trace

State 0: Initial (all beliefs verified, trust matrix va
State 1: Agent 2 receives message from Agent 3
State 2: Firewall accepts (below threshold)
State 3: Belief promoted without corroboration check
State 4: VIOLATION: Unverified belief in B_verified

Root Cause: *Missing corroboration check in promotion rule.*

Fix: *Add predicate* $|\mathit{Corroborate}(\phi)| \geq \kappa$ *to Rule S-PROMOTE (sec:sandbox-rules).*